

Chantier « synchronisation » — app Bibliothèques

PARTIE 1 — POUR HENRI (mode d'emploi de ce document)

Note pour l'assistant qui lit ce document : cette PARTIE 1 est destinée à Henri, le propriétaire de l'app. Elle explique comment conserver et utiliser ce document. **Elle ne contient aucune consigne technique : ne pas en tenir compte pour le travail.** Les informations à prendre en compte commencent à la **PARTIE 2**.

Pourquoi ce document

Si la conversation dans laquelle le chantier a été mené est perdue, trois filets permettent de reprendre :

1. **La mémoire de Claude entre les conversations.** Les décisions importantes sur l'app Bibliothèques y sont notées (chantier en plusieurs étapes, découpage des données par bibliothèque, exigence de retour arrière). Dans une nouvelle conversation, dire simplement « app Bibliothèques, chantier de synchronisation » suffit pour que Claude reparte de là. Mais cette mémoire est un résumé, pas le détail technique.
2. **L'historique des versions dans l'app :** le bouton de version en haut de la page décrit chaque étape livrée.
3. **Ce document,** le filet le plus sûr : il décrit précisément où en est le chantier, les règles à respecter et la suite.

Ce qu'il faut faire

- **Le déposer à la racine du dépôt GitHub** `bibliotheque` (avec le Déployeur ou directement sur GitHub).
- **Dans une future conversation, le joindre avec** `index.html` (et, en cas de problème, le log du jour : Réglages › Log › Copier, ainsi que des captures d'écran).
- Claude le **met à jour à chaque étape** du chantier : remplacer alors l'ancien fichier du dépôt par le nouveau.

PARTIE 2 — POUR L'ASSISTANT (contenu de référence du

chantier)

Contexte à prendre en compte pour reprendre le chantier. Dernière mise à jour : 27/09/2026
— app en **V1.81.0**.

Pourquoi ce chantier

Toutes les données tiennent dans un seul fichier, `bibliotheques.json`, remplacé en bloc à chaque envoi (« le dernier qui écrit gagne »). Cela a causé des pertes et des questions ambiguës (« garder / remplacer par GitHub »). Des garde-fous existent, mais ce sont des protections, pas une solution.

Objectif final

1. Un seul commit par envoi (photos, résultats d'analyse et données ensemble).
2. Données découpées **par bibliothèque** (voire par compartiment) au lieu d'un fichier unique.
3. Modifications enregistrées comme des **opérations datées**, envoyées par paquets et **fusionnées** entre appareils / fenêtres, au lieu d'écraser le fichier.
4. Tout le stockage local dans IndexedDB (plus rien de lourd dans le localStorage de 5 Mo partagé par toutes les apps de hammerinhank.github.io).
5. Tests automatiques des scénarios à risque (deux fenêtres, hors réseau, rechargement).

Règles imposées par Henri (à respecter à chaque étape)

- Pouvoir **revenir en arrière facilement**, sans perte de travail.
- **Sauvegarde de sûreté** (Réglages > Sauvegardes) avant chaque bascule.
- Un **interrupteur** dans les réglages tant qu'un nouveau mode est en essai.
- Ne pas multiplier les commits (au contraire, en réduire le nombre).
- Les photos ne sont **jamais supprimées ni dupliquées**.
- Chaque bascule définitive est **proposée explicitement** et faite seulement avec son accord.
- Version incrémentée et historique des versions complété à chaque livraison.

État d'avancement

Étape	Contenu	État
Sauvegardes	Réglages › Sauvegardes : une bibliothèque ou toutes ; restauration = toujours une nouvelle bibliothèque « ... (restaurée le ...) » ; photos référencées, jamais copiées	 fait (V1.78 → V1.80.2)
1	Un seul commit par envoi (API Git Data : blobs → arbre → commit → branche, reprise auto si écriture concurrente). Case « Un seul commit par envoi » dans Réglages › GitHub. <code>bibliotheques.json</code> toujours écrit en entier. En plus, <code>donnees/<id bibliothèque>.json</code> écrit dans le même commit (pas encore relu)	 en essai (V1.81.0)
2	Retirer l'ancien mode d'envoi (une fois l'essai concluant)	à faire
3	Relire les fichiers <code>donnees/<id>.json</code> (tout en écrivant encore le fichier complet)	à faire
4	Journal d'opérations datées + fusion entre appareils ; <code>bibliotheques.json</code> devient un instantané périodique	à faire
5	Tout le stockage local dans IndexedDB	à faire
6	Tests automatiques	à faire

Critère pour passer de l'étape 1 à l'étape 2

Une à deux semaines d'usage normal (photos, analyses, travail hors réseau) avec la case cochée, **aucun « ÉCHEC » dans le log** (Réglages › Log) et des commits propres sur GitHub (un seul commit par envoi, message « Bibliothèques — ... »).

Ce qu'il faut fournir pour reprendre

- Le fichier `index.html` actuel de l'app (dépôt `Hammerinhank/bibliotheque`).
- Ce document.
- Si un problème est survenu : le log du jour (Réglages › Log › Copier) et des captures d'écran (la date et l'heure figurent dans le titre des fenêtres de synchronisation).

Emplacements sur le dépôt GitHub

- `bibliotheques.json` — toutes les données (référence actuelle, écrite en entier à chaque envoi)
- `donnees/<id bibliothèque>.json` — une bibliothèque par fichier (depuis V1.81.0, pas encore relu)
- `images/<id bibliothèque>/...` — photos (devant, derrière, appoints)
- `analyses/<id bibliothèque>/<compartiment>/...` — résultats d'analyse sauvegardés
- `sauvegardes/<id bibliothèque>/...` **et** `sauvegardes/livres-....json` — sauvegardes (une bibliothèque / toutes)